# problemSet2.r

adria

2026-04-07

```r
##############################################################
# Problem Set 2: Adriano Lama
##############################################################

#-------------------------------------------------------------
#Exercise 1
#-------------------------------------------------------------

# Save time series data from .csv file into a data frame
data <- read.csv2("Asset_Classes.csv", header = TRUE, stringsAsFactors = FALSE)

# Check structure
str(data)
```

```
## 'data.frame':    240 obs. of  6 variables:
##  $ Date      : chr  "29.01.1993" "26.02.1993" "31.03.1993" "30.04.1993" ...
##  $ Stocks    : chr  "0.0103" "0.0818" "0.0133" "-0.0209" ...
##  $ Treasuries: chr  "0.0112" "0.0256" "0.0066" "0.0002" ...
##  $ CorpBonds : chr  "0.0233" "0.023" "0.0035" "0.0077" ...
##  $ RealEstate: chr  "0.005" "0.0067" "0.0093" "0.0044" ...
##  $ Rf        : chr  "0.0072" "0.0072" "0.0069" "0.0065" ...
```

```r
# Convert the first column of the data frame to date format
data$Date <- as.Date(data$Date, "%d.%m.%Y")

# Convert the second column of the data frame to numerical format
for(i in 2:length(data[1,])){data[,i] <- as.numeric(data[,i])}

# Display head and check structure (str) of the data frame
head(data)
```

```
##         Date  Stocks Treasuries CorpBonds RealEstate     Rf
## 1 1993-01-29  0.0103     0.0112    0.0233     0.0050 0.0072
## 2 1993-02-26  0.0818     0.0256    0.0230     0.0067 0.0072
## 3 1993-03-31  0.0133     0.0066    0.0035     0.0093 0.0069
## 4 1993-04-30 -0.0209     0.0002    0.0077     0.0044 0.0065
## 5 1993-05-31  0.0108     0.0043    0.0012     0.0032 0.0065
## 6 1993-06-30  0.0453     0.0126    0.0243     0.0051 0.0065
```

```r
str(data)
```

```
## 'data.frame':    240 obs. of  6 variables:
##  $ Date      : Date, format: "1993-01-29" "1993-02-26" ...
##  $ Stocks    : num  0.0103 0.0818 0.0133 -0.0209 0.0108 0.0453 0.0741 0.0701 -0.0247 0.0578 ...
##  $ Treasuries: num  0.0112 0.0256 0.0066 0.0002 0.0043 0.0126 0.0123 0.0172 0.0103 0.0158 ...
##  $ CorpBonds : num  0.0233 0.023 0.0035 0.0077 0.0012 0.0243 0.0072 0.0249 0.0024 0.005 ...
##  $ RealEstate: num  0.005 0.0067 0.0093 0.0044 0.0032 0.0051 0.0072 0.0107 0.0064 0.0077 ...
##  $ Rf        : num  0.0072 0.0072 0.0069 0.0065 0.0065 0.0065 0.0059 0.0058 0.0057 0.0055 ...
```

```r
#-----------------------------------------------------------
# Exercise 2
#-----------------------------------------------------------


# Load package
#-----------------------------------------------------------
library(quadprog)


# Prepare asset return data
#-----------------------------------------------------------
# Exclude Date column and risk-free rate column
returns <- data[, 2:(ncol(data)-1)]

# Number of assets
N <- length(returns[1,])


# Estimate vector of mean returns
#-----------------------------------------------------------
# Create vector to store annualized mean returns
MU <- rep(NA, N)

# Fill vector with annualized mean returns
for(i in 1:N){
  MU[i] <- round(mean(returns[,i], na.rm = TRUE) * 12, 4)
}
MU
```

```
## [1] 0.0921 0.0596 0.0699 0.0482
```

```r
# Estimate variance-covariance matrix
#-----------------------------------------------------------
# Annualized variance-covariance matrix
SIGMA <- cov(returns, use = "complete.obs") * 12

# Variances found on main diagonal
VAR <- diag(SIGMA)

# Compute return standard deviations (volatilities)
SD <- sqrt(VAR)
```

```r
# Display results
SIGMA
```

```
##                   Stocks    Treasuries    CorpBonds    RealEstate
## Stocks      0.0371005024 -1.427397e-03 1.591744e-03 -1.201199e-04
## Treasuries -0.0014273971  1.114320e-03 7.543526e-04  7.488759e-05
## CorpBonds   0.0015917444  7.543526e-04 3.082668e-03  3.724851e-05
## RealEstate -0.0001201199  7.488759e-05 3.724851e-05  3.097919e-04
```

```r
SD
```

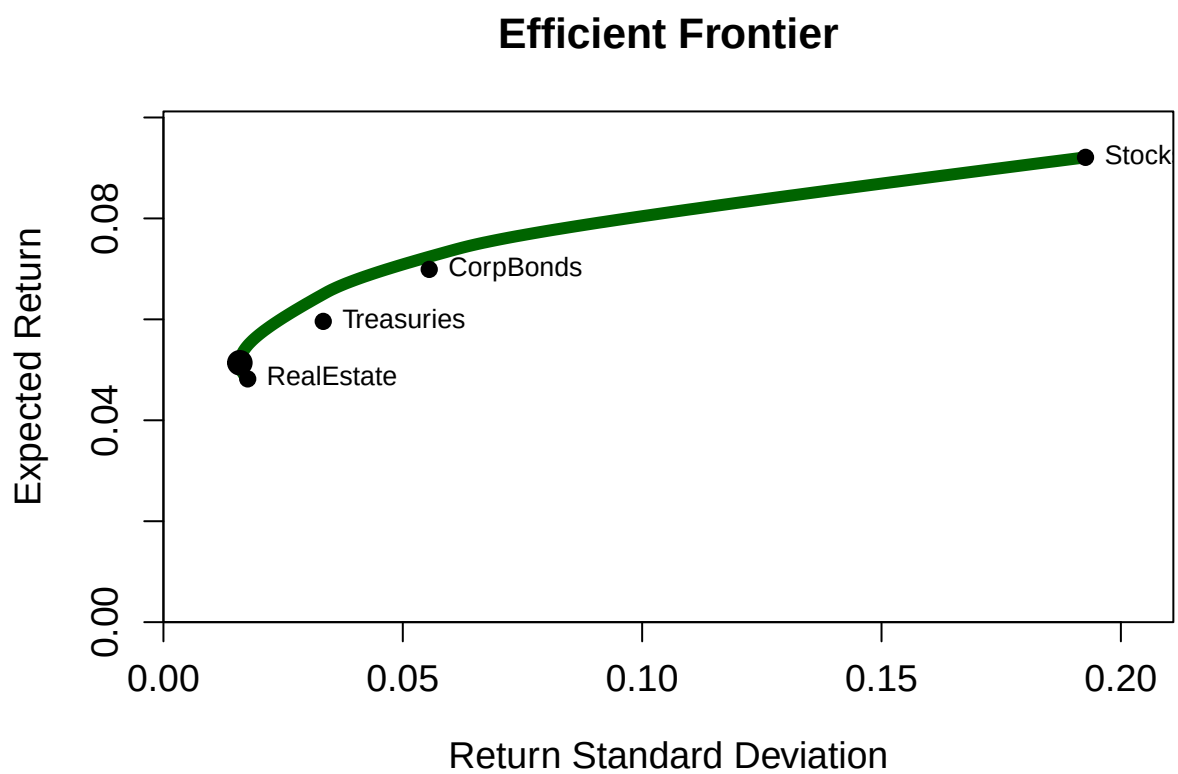
```
##     Stocks Treasuries  CorpBonds RealEstate
## 0.19261491 0.03338143 0.05552178 0.01760091
```

```r
##################
# Markowitz model
##################

#### Prepare objects for optimization function ####
#-------------------------------------------------------------
# Define number and type of constraints
n <- 2 + N      # Total number of constraints
meq <- 2        # First meq constraints treated as equalities

# dvec set to zero (we specify the target function in terms of the variance only)
dvec <- rep(0, N)

# Amat includes left-hand-side values of the constraints
Amat <- matrix(NA, N, n)
Amat[,1] <- MU              # (i)   w1*mu1 + ... + wN*muN = muP  (target return)
Amat[,2] <- rep(1, N)       # (ii)  SUM(w_i) = 1                 (budget constraint)
for(i in 1:N){              # (iii) w_i >= 0                     (short-sale constraints)
  constraint <- rep(0, N)
  constraint[i] <- 1
  Amat[,i+2] <- constraint
}


# Set range of portfolio means along which to minimize variance
#-------------------------------------------------------------
# Use slightly larger step size so the grid does not hit max(MU) exactly
# and solve.QP avoids the degenerate corner solution.
stepsize <- 0.0001
mu_bar <- seq(min(MU), max(MU), by = stepsize)
mu_bar <- mu_bar[mu_bar < max(MU)]


# Create objects to store efficient frontier
#-------------------------------------------------------------
w <- matrix(NA, N, length(mu_bar))        # Matrix for weights
mu <- rep(NA, length(mu_bar))             # Vector for means
sigma <- rep(NA, length(mu_bar))          # Vector for standard deviations
```

```r
#### Optimization for Efficient Frontier ####
#-----------------------------------------------------------
for(i in 1:length(mu_bar)){
  w[,i] <- solve.QP(Dmat = SIGMA,
                    dvec = dvec,
                    Amat = Amat,
                    bvec = c(mu_bar[i], 1, rep(0, N)),
                    meq = meq)$solution
  mu[i] <- t(w[,i]) %*% MU
  sigma[i] <- sqrt(t(w[,i]) %*% SIGMA %*% w[,i])
}


# Find minimum variance portfolio
#-----------------------------------------------------------
min_var <- which.min(sigma)

# Weights of the minimum variance portfolio
w[, min_var]
```

```
## [1] 0.01549310 0.18184041 0.02059319 0.78207330
```

```r
# Show weights of all efficient portfolios
#-----------------------------------------------------------
colnames(w) <- paste0("Portfolio_", 1:ncol(w))
rownames(w) <- colnames(returns)


##########################################
# Plot the efficient frontier
##########################################

#-----------------------------------------------------------
plot(sigma, mu, type = "l",
     lwd = 6, xlim = c(0, max(sigma)*1.1), ylim = c(0, max(mu)*1.1),
     xaxs = "i", yaxs = "i", col = "darkgreen",
     main = "Efficient Frontier",
     xlab = "Return Standard Deviation", ylab = "Expected Return",
     cex.axis = 1.2, cex.lab = 1.2, cex.main = 1.3)

points(SD, MU, pch = 16, cex = 1.2, lwd = 2, col = "black")
points(sigma[min_var], mu[min_var], pch = 19, cex = 1.5, lwd = 2,
       col = "black", bg = "darkgrey")

text(SD, MU, labels = colnames(returns), pos = 4, cex = 0.8)
```




```
#-----------------------------------------------------------
# Exercise 3
# Optimal vs. Naive Diversification
#-----------------------------------------------------------

# Split data into estimation sample and test sample
#-----------------------------------------------------------
data.est <- data[1:120, ]      # January 1993 to December 2002
data.test <- data[121:240, ]   # January 2003 to December 2012

# Use only the four risky asset classes
returns.est <- data.est[, 2:(ncol(data.est)-1)]
returns.test <- data.test[, 2:(ncol(data.test)-1)]


#-----------------------------------------------------------
# Exercise 3a
# Reestimate input parameters and run Markowitz optimization
#-----------------------------------------------------------

# Number of assets
N <- length(returns.est[1,])

# Estimate vector of mean returns
#-----------------------------------------------------------
MU <- rep(NA, N)
```

```r
for(i in 1:length(returns.est[1,])){
  MU[i] <- round(mean(returns.est[,i], na.rm = TRUE) * 12, 4)
}
MU
```

```
## [1] 0.1282 0.0703 0.0763 0.0491
```

```r
# Estimate variance-covariance matrix
#-----------------------------------------------------------
SIGMA <- cov(returns.est, use = "complete.obs") * 12

VAR <- diag(SIGMA)
SD <- sqrt(VAR)

SIGMA
```

```
##                   Stocks    Treasuries    CorpBonds    RealEstate
## Stocks      4.210585e-02 -8.206563e-05 0.0005188384 -7.042071e-05
## Treasuries -8.206563e-05  1.061065e-03 0.0009499945  1.605351e-04
## CorpBonds   5.188384e-04  9.499945e-04 0.0022834015  1.223466e-04
## RealEstate -7.042071e-05  1.605351e-04 0.0001223466  8.266345e-05
```

```r
SD
```

```
##      Stocks  Treasuries   CorpBonds  RealEstate
## 0.205197089 0.032573986 0.047784951 0.009091944
```

```r
##################
# Markowitz model
##################

#### Prepare objects for optimization function ####
#-----------------------------------------------------------
n <- 2 + N
meq <- 2
dvec <- rep(0, N)

Amat <- matrix(NA, N, n)
Amat[,1] <- MU
Amat[,2] <- rep(1, N)
for(i in 1:N){
  constraint <- rep(0, N)
  constraint[i] <- 1
  Amat[,i+2] <- constraint
}


# Set range of portfolio means along which to minimize variance
#-----------------------------------------------------------
stepsize <- 0.0001
mu_bar <- seq(min(MU), max(MU), by = stepsize)
```

```r
mu_bar <- mu_bar[mu_bar < max(MU)]


# Create objects to store efficient frontier
#-----------------------------------------------------------
w <- matrix(NA, N, length(mu_bar))
mu <- rep(NA, length(mu_bar))
sigma <- rep(NA, length(mu_bar))


#### Optimization for Efficient Frontier ####
#-----------------------------------------------------------
for(i in 1:length(mu_bar)){
  w[,i] <- solve.QP(Dmat = SIGMA,
                    dvec = dvec,
                    Amat = Amat,
                    bvec = c(mu_bar[i], 1, rep(0, N)),
                    meq = meq)$solution
  mu[i] <- t(w[,i]) %*% MU
  sigma[i] <- sqrt(t(w[,i]) %*% SIGMA %*% w[,i])
}


# Find minimum variance portfolio
#-----------------------------------------------------------
min_var <- which.min(sigma)

# Weights of the minimum variance portfolio
w[, min_var]
```

```
## [1] 3.792668e-03 2.859976e-19 0.000000e+00 9.962073e-01
```

```r
#-----------------------------------------------------------
# Exercise 3b
# 1/N strategy in the estimation sample
#-----------------------------------------------------------

# Equal weights for all four asset classes
w.eq <- rep(1/N, N)

# Compute return time series of the 1/N portfolio
ret.eq <- rep(NA, nrow(returns.est))
for(i in 1:nrow(returns.est)){
  ret.eq[i] <- t(w.eq) %*% as.numeric(returns.est[i, ])
}

# Sample mean and sample standard deviation of historical returns
mean.eq <- mean(ret.eq)
sd.eq <- sd(ret.eq)

mean.eq
```

```
## [1] 0.006750417
```

```r
sd.eq
```

```
## [1] 0.0159314
```

```r
# Annualize 1/N portfolio statistics for the plot
#-----------------------------------------------------------
mu.eq <- mean.eq * 12
sigma.eq <- sd.eq * sqrt(12)

mu.eq
```

```
## [1] 0.081005
```

```r
sigma.eq
```

```
## [1] 0.05518799
```

```r
#-----------------------------------------------------------
# Exercise 3c
# Find efficient portfolio with same standard deviation as 1/N
#-----------------------------------------------------------

same_sd <- which.min(abs(sigma.eq - sigma))

# Mean, standard deviation, and weights of the corresponding efficient portfolio
mu[same_sd]
```

```
## [1] 0.086
```

```r
sigma[same_sd]
```

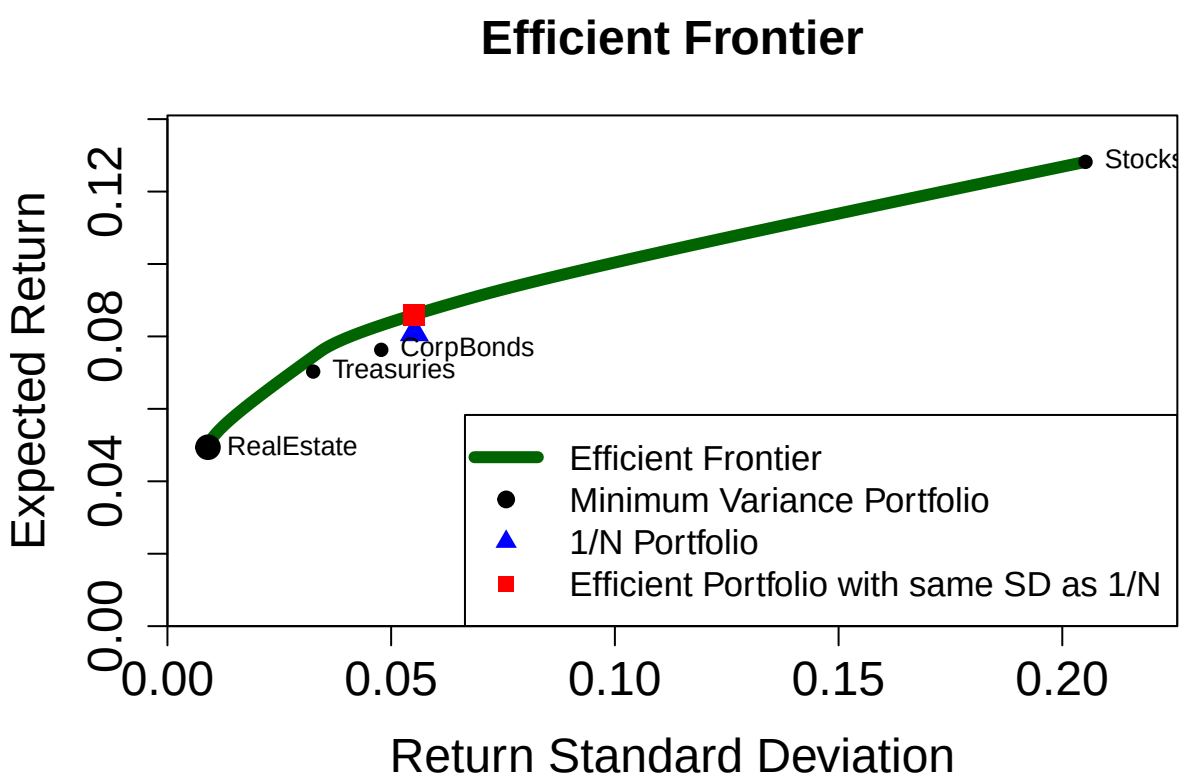
```
## [1] 0.05513652
```

```r
w[, same_sd]
```

```
## [1]  2.113459e-01  2.114756e-01  5.771785e-01 -5.551115e-17
```

```r
##########################################
# Plot the efficient frontier
##########################################

#-----------------------------------------------------------
plot(sigma, mu, type = "l",
     lwd = 6, xlim = c(0, max(c(sigma, SD, sigma.eq))*1.1),
     ylim = c(0, max(c(mu, MU, mu.eq))*1.1),
     xaxs = "i", yaxs = "i", col = "darkgreen",
     main = "Efficient Frontier",
     xlab = "Return Standard Deviation", ylab = "Expected Return",
     cex.axis = 1.5, cex.lab = 1.5, cex.main = 1.5)
```

```r
# Asset class portfolios
points(SD, MU, pch = 16, cex = 1, lwd = 2, col = "black")

# Minimum variance portfolio
points(sigma[min_var], mu[min_var], pch = 19, cex = 1.5, lwd = 2,
       col = "black", bg = "darkgrey")

# 1/N portfolio
points(sigma.eq, mu.eq, pch = 17, cex = 1.5, lwd = 2, col = "blue")

# Efficient portfolio with same standard deviation as 1/N
points(sigma[same_sd], mu[same_sd], pch = 15, cex = 1.5, lwd = 2, col = "red")

# Labels for the four asset classes
text(SD, MU, labels = colnames(returns.est), pos = 4, cex = 0.8)

legend("bottomright",
       c("Efficient Frontier",
         "Minimum Variance Portfolio",
         "1/N Portfolio",
         "Efficient Portfolio with same SD as 1/N"),
       lty = c(1, NA, NA, NA),
       pch = c(NA, 19, 17, 15),
       lwd = c(6, NA, NA, NA),
       col = c("darkgreen", "black", "blue", "red"),
       cex = 1.1)
```




```
#-------------------------------------------------------------
# Exercise 3d
# Out-of-sample Sharpe Ratios in the test period
#-------------------------------------------------------------

# Risk-free rate in the test sample
#-------------------------------------------------------------
rf.test <- data.test[, ncol(data.test)]


# Compute realized returns of the 1/N portfolio in the test sample
#-------------------------------------------------------------
ret.eq.test <- rep(NA, nrow(returns.test))

for(i in 1:nrow(returns.test)){
  ret.eq.test[i] <- t(w.eq) %*% as.numeric(returns.test[i, ])
}

# Check realized returns
head(ret.eq.test)
```

```
## [1] -0.009275 -0.003150 -0.012300  0.042900  0.017150  0.008925
```

```
# Compute realized returns of the efficient portfolio in the test sample
#-------------------------------------------------------------
```

```r
ret.eff.test <- rep(NA, nrow(returns.test))

for(i in 1:nrow(returns.test)){
  ret.eff.test[i] <- t(w[, same_sd]) %*% as.numeric(returns.test[i, ])
}

# Check realized returns
head(ret.eff.test)
```

```
## [1] -0.007042579  0.003844540 -0.010400218  0.042759454  0.026143240
## [6]  0.006597339
```

```r
# Compute out-of-sample excess returns
#-----------------------------------------------------------
excess.eq <- ret.eq.test - rf.test
excess.eff <- ret.eff.test - rf.test

# Check excess returns
head(excess.eq)
```

```
## [1] -0.011675 -0.005350 -0.014400  0.040800  0.015150  0.007125
```

```r
head(excess.eff)
```

```
## [1] -0.009442579  0.001644540 -0.012500218  0.040659454  0.024143240
## [6]  0.004797339
```

```r
# Compute mean of out-of-sample excess returns
#-----------------------------------------------------------
mu_hat_e_eq <- mean(excess.eq)
mu_hat_e_eff <- mean(excess.eff)

mu_hat_e_eq
```

```
## [1] 0.002755
```

```r
mu_hat_e_eff
```

```
## [1] 0.003166481
```

```r
# Compute standard deviation of out-of-sample excess returns
#-----------------------------------------------------------
sigma_hat_e_eq <- sd(excess.eq)
sigma_hat_e_eff <- sd(excess.eff)

sigma_hat_e_eq
```

```
## [1] 0.01436251
```

```r
sigma_hat_e_eff
```

```
## [1] 0.01697166
```

```r
# Compute out-of-sample Sharpe Ratios
#-----------------------------------------------------------
SR.eq <- mu_hat_e_eq / sigma_hat_e_eq
SR.eff <- mu_hat_e_eff / sigma_hat_e_eff

SR.eq
```

```
## [1] 0.1918188
```

```r
SR.eff
```

```
## [1] 0.1865746
```

```r
# Report: Out-of-sample Sharpe Ratios
```

```r
# 1/N Portfolio Sharpe Ratio:       0.1918
# Efficient Portfolio Sharpe Ratio:  0.1866
#
# Observation: The naïve 1/N portfolio achieves a slightly higher
# out-of-sample Sharpe Ratio (0.192 vs. 0.187) than the Markowitz-
# optimized efficient portfolio, despite the latter dominating the
# 1/N portfolio in the estimation sample (higher expected return
# for the same level of risk).
#
# Explanation: This result is driven by estimation risk. The
# Markowitz model treats estimated means, variances and
# covariances as if they were the true population parameters.
# In practice, especially expected returns are estimated with
# considerable error from historical data. The optimizer exploits
# these estimation errors, tilting weights heavily toward assets
# that happened to perform well in the estimation period (here:
# ~58% in corporate bonds, ~21% each in stocks and treasuries,
# 0% in real estate). When the market environment changes in
# the test period (2003-2012), these concentrated bets do not
# pay off as expected and the out-of-sample risk increases
# (sigma_hat = 0.0170 vs. 0.0144 for the 1/N portfolio).
# The 1/N strategy by contrast, is parameter-free and therefore
# immune to estimation error. Its equal weighting provides
# broader diversification, which proves more robust when the
# future deviates from the past. This is consistent with the
# broader insight that, when the number of assets is small
# and the estimation window is limited, naïve diversification
# can match or even outperform mean-variance optimization
# on a risk-adjusted basis out of sample.
```